



Competitive Programming: Exhaustive Search Algorithm

Presenter: Dr. Ha Viet Uyen Synh.



Warm up #1. Count set bits in an integer

Write an efficient program to count number of 1s in binary representation of an integer.

Ex:

Input : $n = 6$

Output : 2

Binary representation of 6 is 110
and has 2 set bits

Input : $n = 13$

Output : 3

Binary representation of 13 is 1101
and has 3 set bits



Warm up #2. COPRIME

Xét A là dãy các số nguyên tố

2, 3, 5, 7, 11, 13, 17, 19, ...

và dãy B gồm các số thu được từ dãy A bằng cách ghép hai số liên tiếp trong A:

23, 57, 1113, 1719, ...

Trong dãy B có những phần tử là số nguyên tố. Chẳng hạn 23, 3137, 8389, 157163, ...

Các số nguyên tố trong dãy B ta gọi là số nguyên tố ghép. Dãy con của dãy B bao gồm những số nguyên trong B được giữ nguyên thứ tự của chúng trong B sẽ được gọi là dãy số nguyên tố ghép và ký hiệu dãy này là C.

Yêu cầu: Cho trước số nguyên dương $N \leq 500$, hãy tìm số hạng thứ N của dãy C.

Dữ liệu: Nhập từ văn bản COPRIME.INP chứa số N.

Kết quả: Xuất ra trên một dòng của tập tin văn bản COPRIME.OUT số hạng thứ N của dãy số nguyên tố ghép.

Ví dụ:

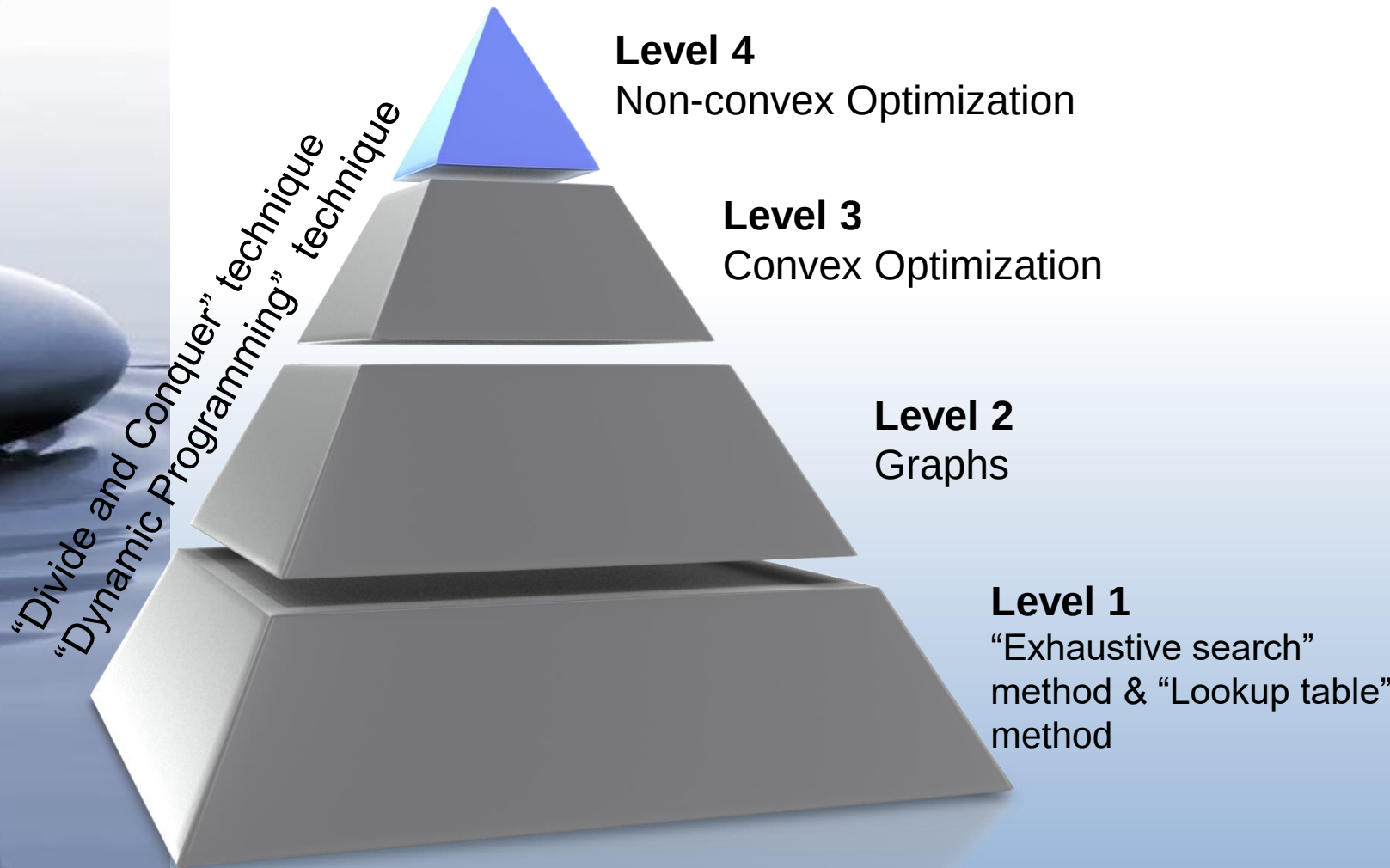
COPRIME.INP

1

COPRIME.OUT

23

How to solve a problem in computer science?





Part A

EXHAUSTIVE SEARCH



Exhaustive Search algorithm

Keywords: brute-force search, exhaustive search, exhausted check, generate and test

In computer science, exhaustive search consists of systematically enumerating all possible candidates for the solution and checking whether each candidate satisfies the problem's statement.

While an exhaustive search is simple to implement, and will always find a solution if it exists, its cost is proportional to the number of candidate solutions – which in many practical problems tends to grow very quickly as the size of the problem increases.

Exhaustive search should not be confused with backtracking, where large sets of solutions can be discarded without being explicitly enumerated.

* In competitive programming, exhaustive search is used to create test cases.

A stack of smooth, dark stones is shown on the left side of the image, resting on a reflective surface. The stones are stacked horizontally, with the top stone being the most prominent. The background is a soft, out-of-focus blue and white, suggesting a calm body of water or a misty sky.

```
// Naïve solution
#include <bits/stdc++.h>

// function to print the divisors
void printDivisors(int n)
{
    for (int i=1;i<=n;i++)
        if (n%i==0)
            printf("%d ",i);
}

/* Driver program to test above function */
int main()
{
    printf("The divisors of 100 are: n");
    printDivisors(100);
    return 0;
}
```



```
// A Better (than Naive) Solution to find all divisors
#include <bits/stdc++.h>
```

```
// Function to print the divisors
void printDivisors(int n) {
    // Note that this loop runs till square root
    for (int i=1; i<=sqrt(n)+1; i++)    {
        if (n%i==0)        {
            // If divisors are equal, print only one
            if (n/i == i)
                printf("%d ", i);
            else // Otherwise print both
                printf("%d %d ", i, n/i);
        }
    }
}
```

```
/* Driver program to test above function */
int main() {
    printf("The divisors of 100 are: n");
    printDivisors(100);
    return 0;
}
```

A vertical stack of five smooth, dark, rounded stones sits on a calm, reflective surface. The stones are stacked in a slightly offset manner, creating a sense of balance and harmony. The surface of the water reflects the stones and the light from the sky, which is a pale, hazy blue. The overall mood is serene and minimalist.

Example #1

To find the divisors of a natural number n , exhaustive search would enumerate all integers from 1 to n , and check whether each of them divides n without remainder.

Input : $n = 10$

Output: 1 2 5 10

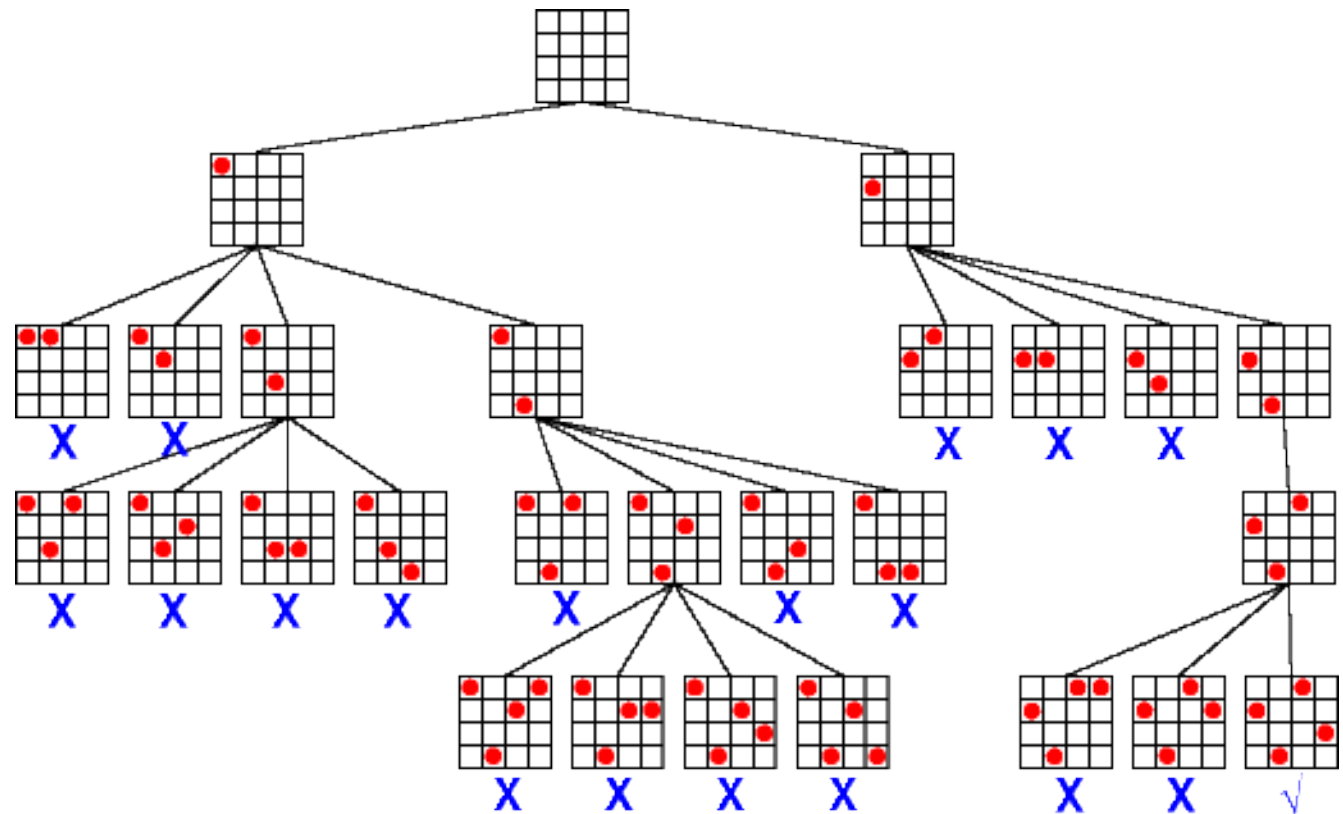
Input: $n = 100$

Output: 1 2 4 5 10 20 25 50 100

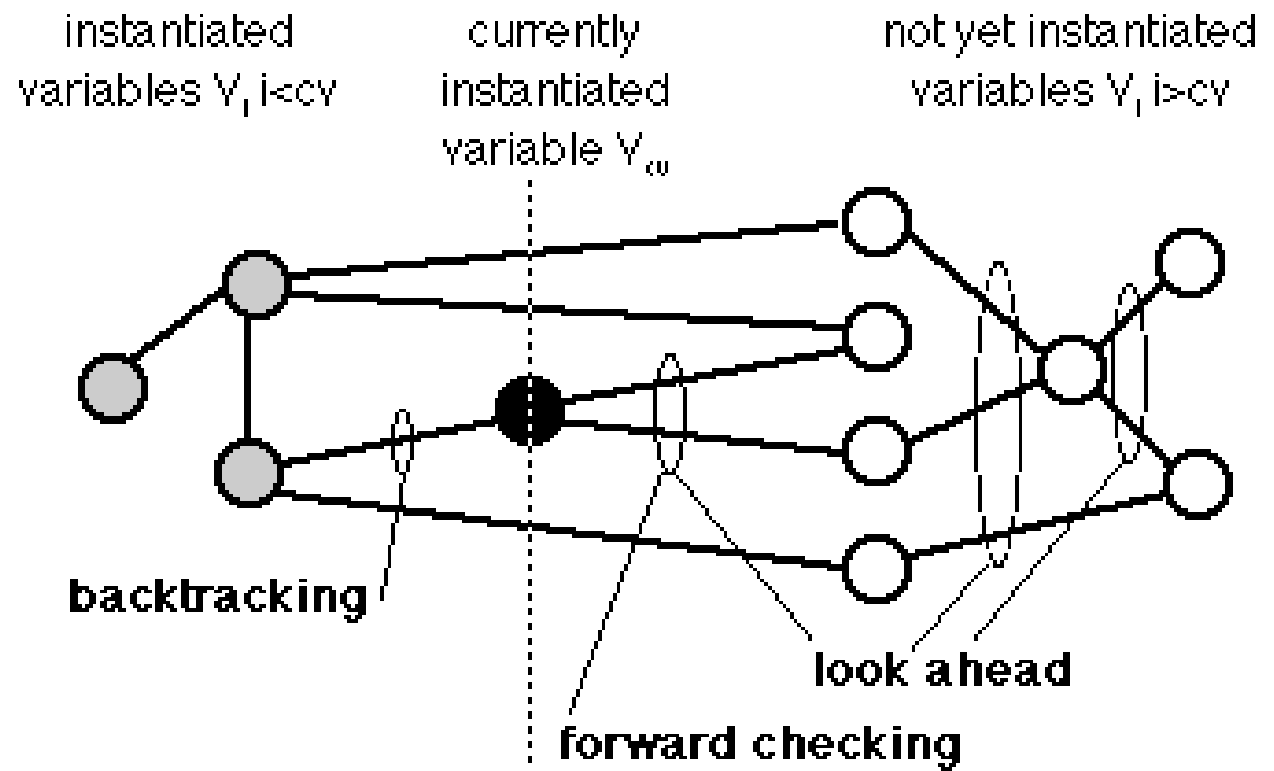
Input: $n = 125$

Output: 1 5 25 125

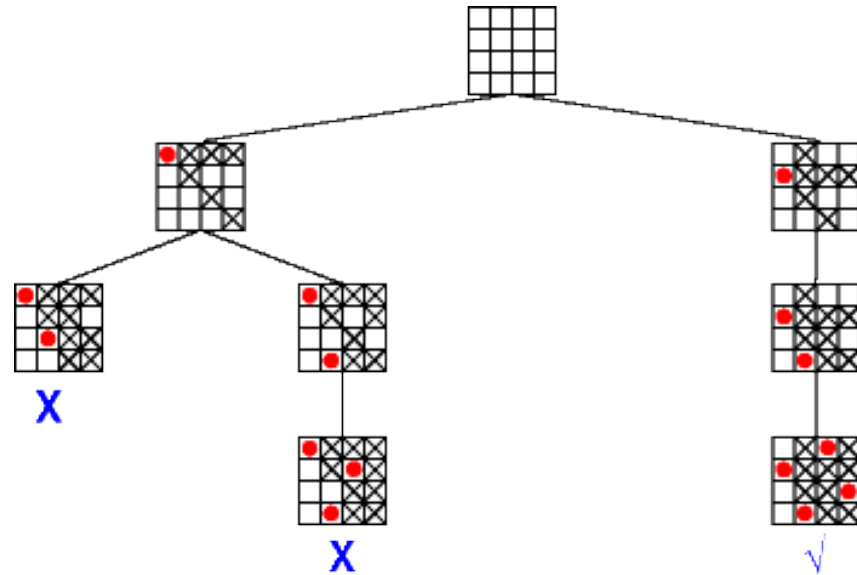
Example 2: The 4 queens puzzle



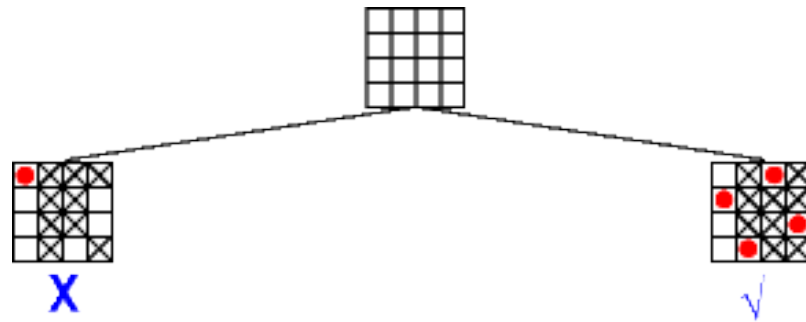
Propagation techniques



Example _ Forward Checking



Example _ Look Ahead



Practical approaches



$\begin{array}{c} \diagup X \\ Y \diagdown \end{array}$		undetermined	determined
$\begin{array}{c} \text{undetermined} \\ \text{determined} \end{array}$	undetermined	Backtracking (Recursion)	Backtracking (Recursion)
	determined	Forward Checking Y loop statements	Look Ahead Only 1 loop statement

Y : number of positions changed (determined & undetermined)

X : number of states at each position (determined & undetermined)

Q.124

Cho số tự nhiên n. hãy đặt các dấu + hoặc - vào giữa các chữ số 1, 2, 3, 4, 5, 6, 7, 8, 9 viết theo thứ tự đã cho để tạo ra 1 biểu thức có giá trị bằng n.

VD : $n=122 \Rightarrow 12 + 34 - 5 - 6 + 78 + 9$

buf [] = "+1- 2 3+4-5 6+7 8-9"

buf2[] = "+1-23+4-56+78-9" $\rightarrow$ "120120102"

int a[9];

n = sscanf(buf2, "%d%d%d%d%d%d%d%d", &a[0], &a[1], &a[2], &a[3], &a[4], &a[5], &a[6], &a[7], &a[8]);

scanf

sscanf(buf, formatstring,)

fscanf



itoa, ltoa, ftoa,

atoi , atoll, atof,

$N = 3^9$

index= 324; nbuf="112021"=>nbuf2[]="000112021"

itoa(index, buf, 3)



Part B

LOOK-UP TABLE

What Is a Lookup Table?

A lookup table or lookup file holds static data and is used to look up a secondary value based on a primary value. It can be used to translate encoded information into a user-friendly description, to validate input values by matching them to a list of valid items, or to translate a shorthand entry into something more detailed.

Large Data Table

id	column1	column2	column3	column4	column5
0	data	data	data	Entry1	data
1	data	data	data	Entry2	data
2	data	data	data	Entry2	data
3	data	data	data	Entry3	data
4	data	data	data	Entry2	data
5	data	data	data	Entry1	data



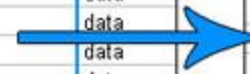
Automatically create
Lookup Table

Normalized Data Table

id	column1	column2	column3	column4_id	column5
0	data	data	data	0	data
1	data	data	data	1	data
2	data	data	data	1	data
3	data	data	data	2	data
4	data	data	data	1	data
5	data	data	data	0	data

Lookup Table

id	column4
0	Entry1
1	Entry2
2	Entry3





P1. Count set bits in an integer

Write an efficient program to count number of 1s in binary representation of an integer.

Ex:

Input : $n = 6$

Output : 2

Binary representation of 6 is 110
and has 2 set bits

Input : $n = 13$

Output : 3

Binary representation of 13 is 1101
and has 3 set bits



Solution

Here we assume that the size of INT is 32-bits. It's hard to count all 32 bits in one go using lookup table (" because it's infeasible to create lookup table of size $2^{32}-1$ "). **So we break 32 bits into 8 bits of chunks**(now lookup table of size (2^8-1) index : 0-255).

Let's number be : 354

in Binary : 000000000000000000000000101100010

Split it into 8 bits chunks :

In Binary : 00000000 | 00000000 | 00000001 | 01100010

In decimal : 0 0 1 98

Now Count Set_bits using LookupTable

LookupTable[0] = 0

LookupTable[1] = 1

LookupTable[98] = 3

so Total bits count : 4



```
#include <iostream>
using namespace std;

// Generate a lookup table for 32 bit integers
#define B2(n) n, n + 1, n + 1, n + 2
#define B4(n) B2(n) \, B2(n + 1), B2(n + 1), B2(n + 2)
#define B6(n) B4(n) \, B4(n + 1), B4(n + 1), B4(n + 2)

// Lookup table that store the reverse of each table
unsigned int lookuptable[256] = { B6(0), B6(1), B6(1), B6(2) };

// function countset Bits Using lookup table
// ans return set bits count
unsigned int countSetBits(int N)
{
    // first chunk of 8 bits from right
    unsigned int count = lookuptable[N & 0xff] +

                                // second chunk from right
                                lookuptable[(N >> 8) & 0xff] +

                                // third and fourth chunks
                                lookuptable[(N >> 16) & 0xff] +
                                lookuptable[(N >> 24) & 0xff];

    return count;
}

int main()
{
    // generate lookup table
    for (int i = 0; i < 256; i++)
        lookuptable[i] = (i & 1) + lookuptable[i / 2];

    unsigned int N = 354;
    cout << countSetBits(N) << endl;

    return 0;
} //Output: 4
```



P2. COPRIME

Xét A là dãy các số nguyên tố

2, 3, 5, 7, 11, 13, 17, 19, ...

và dãy B gồm các số thu được từ dãy A bằng cách ghép hai số liên tiếp trong A:

23, 57, 1113, 1719, ...

Trong dãy B có những phần tử là số nguyên tố. Chẳng hạn 23, 3137, 8389, 157163, ...

Các số nguyên tố trong dãy B ta gọi là số nguyên tố ghép. Dãy con của dãy B bao gồm những số nguyên trong B được giữ nguyên thứ tự của chúng trong B sẽ được gọi là dãy số nguyên tố ghép và ký hiệu dãy này là C.

Yêu cầu: Cho trước số nguyên dương $N \leq 500$, hãy tìm số hạng thứ N của dãy C.

Dữ liệu: Nhập từ văn bản COPRIME.INP chứa số N.

Kết quả: Xuất ra trên một dòng của tập tin văn bản COPRIME.OUT số hạng thứ N của dãy số nguyên tố ghép.

Ví dụ:

COPRIME.INP

1

COPRIME.OUT

23



PART A

BACKTRACKING

Backtracking

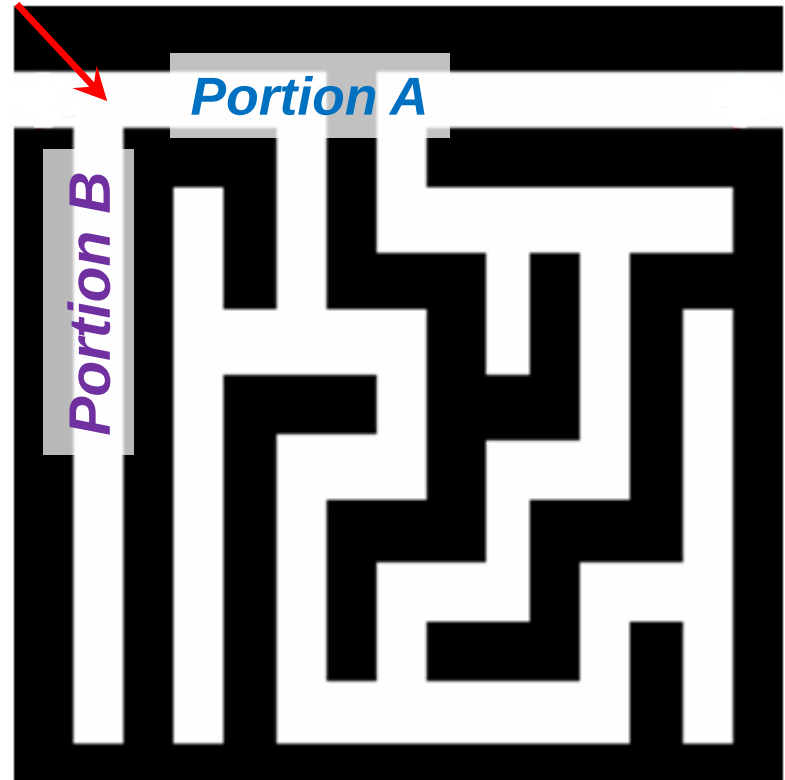
Backtracking is a technique used to solve problems with a large search space, by systematically trying and eliminating possibilities.

A standard example of backtracking would be going through a maze. At some point in a maze, you might have two options of which direction to go:

Junction

Portion A

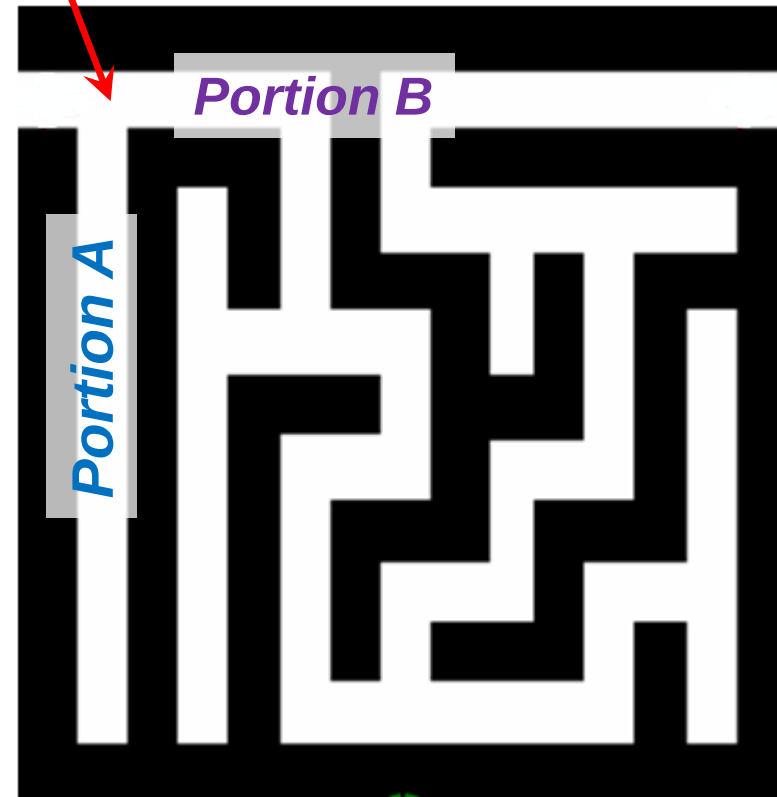
Portion B



Backtracking

- One strategy would be to try going through **Portion A** of the maze.
 - If you get stuck before you find your way out, then you *"backtrack"* to the junction.
- At this point in time you know that **Portion A** will *NOT* lead you out of the maze,
 - so you then start searching in **Portion B**

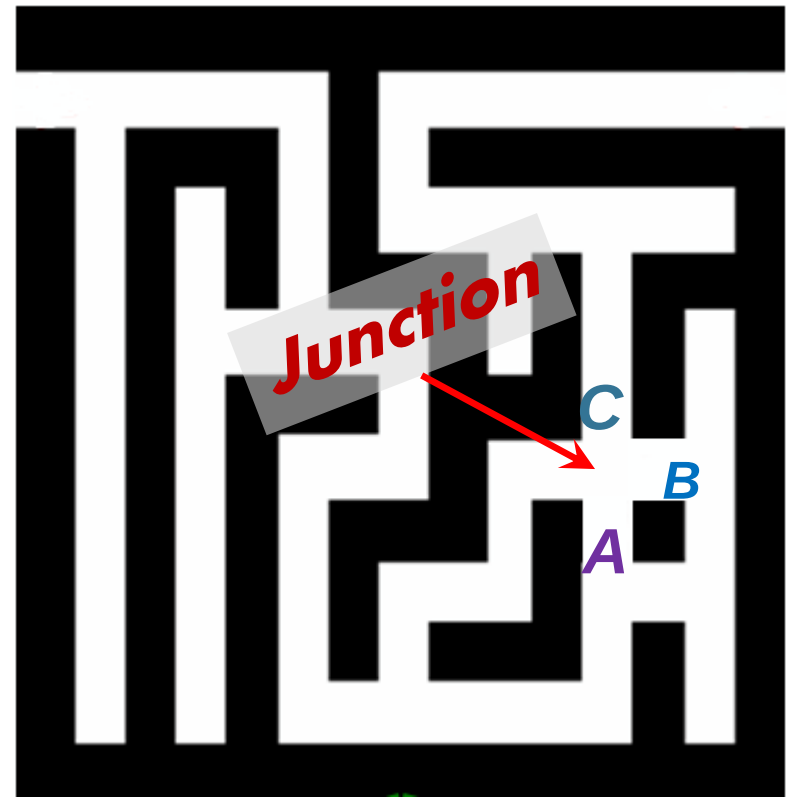
Junction



Backtracking

Clearly, at a single junction you could have even more than 2 choices. The backtracking strategy says to try each choice, one after the other, if you ever get stuck, "**backtrack**" to the junction and try the next choice.

If you try all choices and never found a way out, then there IS no solution to the maze.



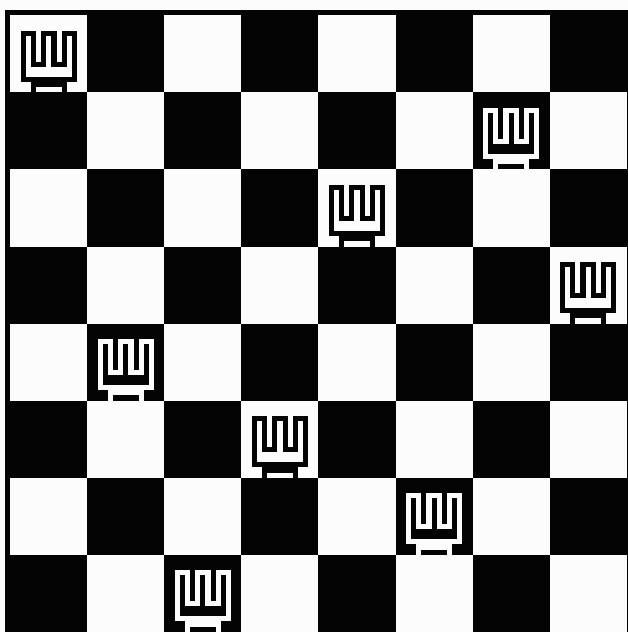


Backtracking – Eight Queens Problem

Find an arrangement of **8** queens on a single chess board such that no two queens are attacking one another.

In chess, queens can move all the way down any row, column or diagonal (so long as no pieces are in the way).

Due to the first two restrictions, it's clear that each row and column of the board will have exactly one queen.



Backtracking – Eight Queens Problem

The backtracking strategy is as follows:

- 1) Place a queen on the first available square in row 1.
- 2) Move onto the next row, placing a queen on the first available square there (that doesn't conflict with the previously placed queens).
- 3) Continue in this fashion until either:
 - a) you have solved the problem, or
 - b) you get stuck.

When you get stuck, remove the queens that got you there, until you get to a row where there is another valid square to try.





Backtracking Algorithm

Based on depth-first recursive search

Approach

1. Tests whether solution has been found
2. If found solution, return it
3. Else for each choice that can be made
 - a) Make that choice
 - b) Recursion
 - c) If recursion returns a solution, return it
4. If no choices remain, return failure

Some times called “search tree”

Recursive Thinking

Recursion is a programming technique in which a method can call itself in order to fulfill its purpose

A recursive definition is one which uses the word or concept being defined in the definition itself

In some situations, a recursive definition can be an appropriate way to express a concept





Fund Raising: Iteration vs. Recursion

Problem: Collect \$1,000.00 for charity

Assumption: Everyone is willing to donate a penny

Iterative Solution

Visit 100,000 people, asking each for a penny

Recursive Solution

If you are asked to collect a penny, give a penny to the person who asked you for it

Otherwise

Visit 10 people and ask them to each raise $\frac{1}{10}$ th of the amount of money that you have been asked to raise

Collect the money that they give you and combine it into one bag

Give it to the person who asked you to collect the money



General Form of Recursive methods

Solve(Problem)

```
{  
  if (Problem is minimal/not decomposable)  
    solve Problem directly.  
  else {  
    (1) Decompose Problem into one or more similar, strictly  
        smaller subproblems: SP1,... ,SPN  
    (2) Recursively call Solve (this method) on each  
        subproblem: Solve(SP1), ..., Solve(SPN)  
    (3) Combine the solutions to these subproblems into a  
        solution that solves the original Problem  
  }  
}
```



Recursion in Math

Mathematical formulas are often expressed recursively
 $N!$, for any positive integer N , is defined to be the product of all integers between 1 and N inclusive

This definition can be expressed recursively:

$$\begin{aligned} 1! &= 1 \\ N! &= N * (N-1)! \end{aligned}$$

A factorial is defined in terms of another factorial until the base case of $1!$ is reached



Pow: Raising a Number to a Power

$$A^N = \begin{cases} 1 & \text{if } N = 0 \\ A * A^{N-1} & \text{if } N > 0 \end{cases}$$

```
double pow(double a, int n)
{
    if (n == 0)
        return 1.;
    else
        return a*pow(a,n-1)
}
```

Example using Definition

$$A^4 = A * A^3$$

$$= A * A * A^2$$

$$= A * A * A * A^1$$

$$= A * A * A * A * A^0$$

$$= A * A * A * A * 1$$



Example #1

Consider the problem of computing the sum of all the numbers between 1 and N, inclusive

If N is 5, the sum is

$$1 + 2 + 3 + 4 + 5$$

This problem can be expressed recursively as:

$$\text{sum}(N) = N + \text{sum}(N-1)$$

$$\sum_{i=1}^N i = N + \sum_{i=1}^{N-1} i$$

$$= N + N - 1 + \sum_{i=1}^{N-2} i$$

$$= N + N - 1 + N - 2 + \sum_{i=1}^{N-3} i$$

$$= N + N - 1 + N - 2 + \cdots + 2 + 1$$

A stack of smooth, dark stones is positioned on the left side of the image, resting on a highly reflective surface that creates a clear mirror image of the stones. The background is a soft, light blue gradient.

Recursion vs. Iteration

Just because we can use recursion to solve a problem, doesn't mean we should

For instance, we usually would not use recursion to solve the sum of 1 to N

The iterative version is easier to understand (in fact there is a formula that is superior to both recursion and iteration in this case)

You must be able to determine when recursion is the correct technique to use

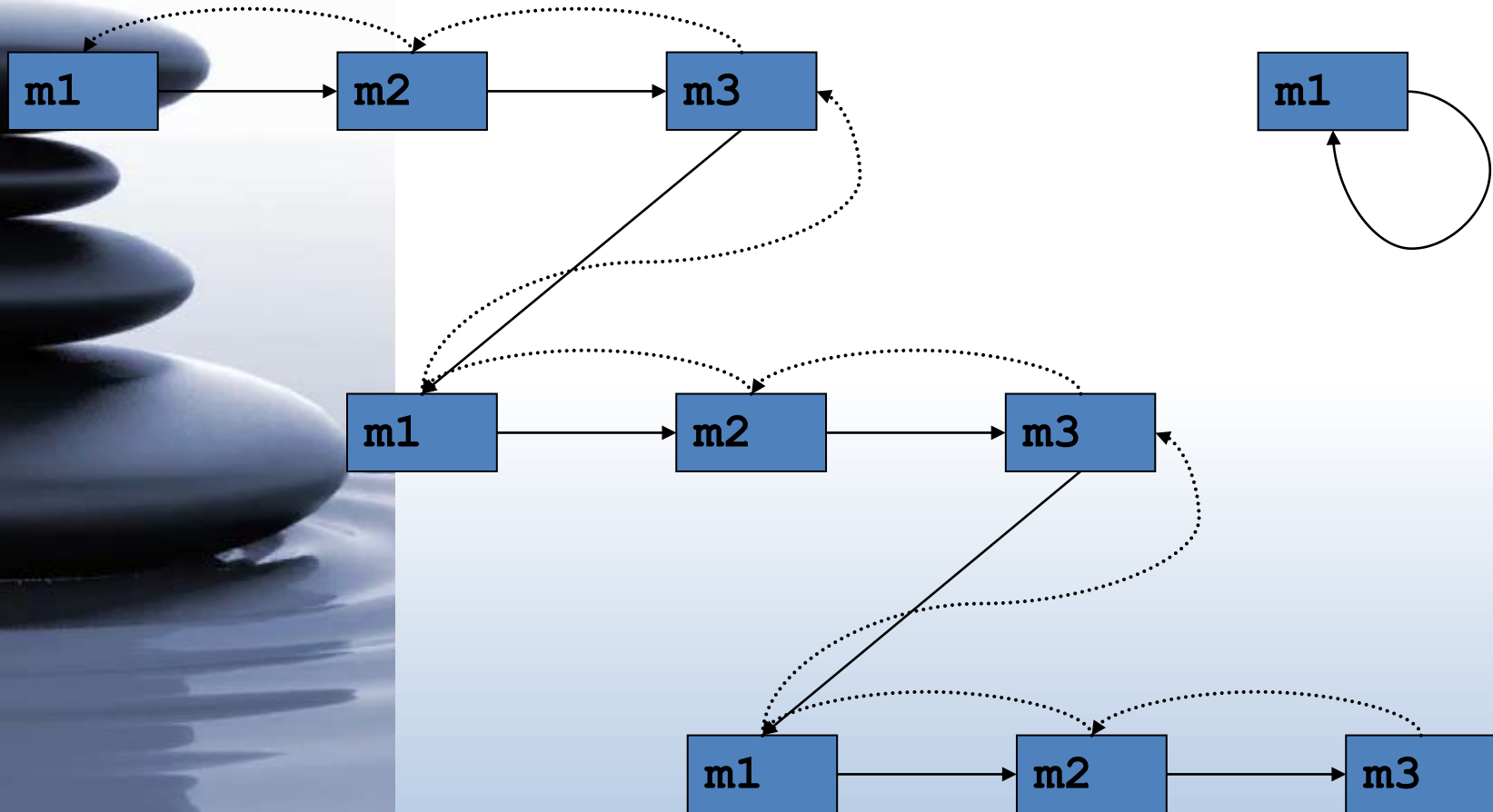
Every recursive solution has a corresponding iterative solution

For example, the sum of the numbers between 1 and N can be calculated with a loop

Recursion has the overhead of multiple method invocations

However, for some problems recursive solutions are often more simple and elegant than iterative solutions

Direct vs. Indirect Recursion



A stack of smooth, dark stones is positioned on the left side of the slide, resting on a reflective surface that shows their reflection. The stones are stacked horizontally, with the top stone being the most prominent. The background is a light, neutral color.

Problem #1: Maze Traversal

Let's use recursion to find a path through a maze

A path can be found through a maze from location x if a path can be found from any of the locations neighboring x

We can mark each location we encounter as “visited” and then attempt to find a path from that location's unvisited neighbors

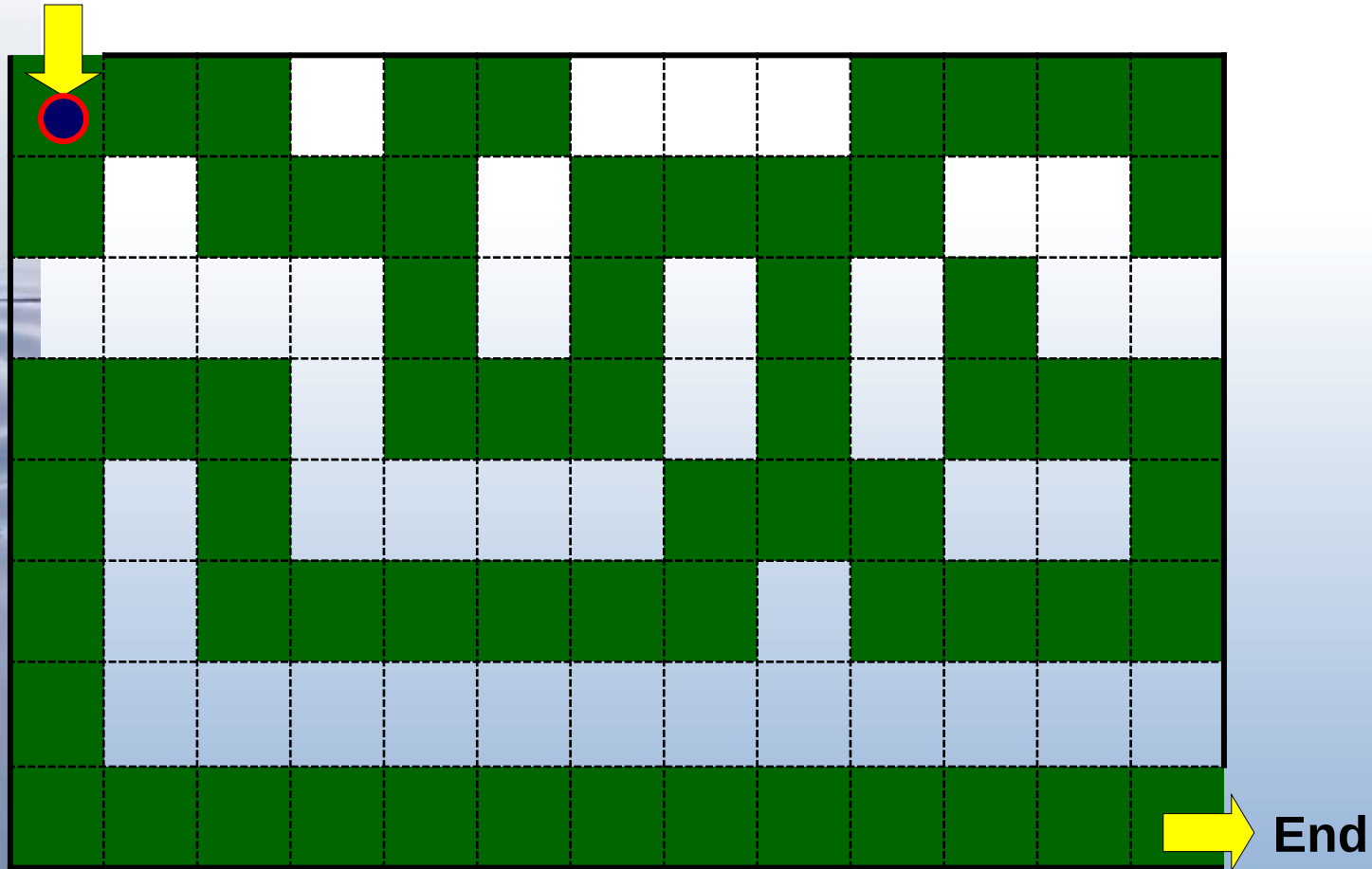
Recursion will be used to keep track of the path through the maze using the run-time stack

The base cases are

a prohibited (blocked) move, or
arrival at the final destination

Solving a Maze

Start



Maze.cpp

```
#include <stdio.h>

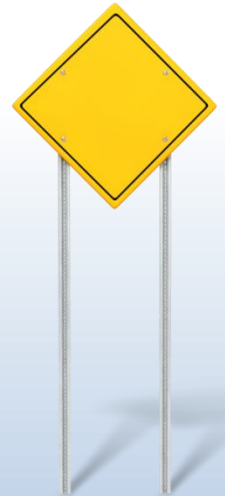
#define TRIED          3
#define PATH           7

int grid [8][13] = {
    {1,1,1,0,1,1,0,0,0,1,1,1,1},
    {1,0,1,1,1,0,1,1,1,1,0,0,1},
    {0,0,0,0,1,0,1,0,1,0,1,0,0},
    {1,1,1,0,1,1,1,0,1,0,1,1,1},
    {1,0,1,0,0,0,0,1,1,1,0,0,1},
    {1,0,1,1,1,1,1,1,0,1,1,1,1},
    {1,0,0,0,0,0,0,0,0,0,0,0,0},
    {1,1,1,1,1,1,1,1,1,1,1,1,1} };

int traverse (int row, int column)
{
    boolean done = false;
    if (valid (row, column))
    {
        // this cell has been tried
        grid[row][column] = TRIED;
        if (row == grid.length-1 &&
            column == grid[0].length-1)
            done = true; // the maze is solved
        else {
            // down
            done = traverse (row+1, column);
            if (!done) // right
                done = traverse (row, column+1);
            if (!done) // up
                done = traverse (row-1, column);
            if (!done) // left
                done = traverse (row, column-1);
        }
        // this location is part of the final path
        if (done)
            grid[row][column] = PATH;
    }
    return done;
}
```

```
int valid (int row, int column)
{
    boolean result = false;
    // check if cell is in the bounds of the matrix
    if (row >= 0 && row < grid.length && column >= 0 && column <
        grid[row].length)
        // check if cell is not blocked and not previously tried
        if (grid[row][column] == 1)
            result = true;
    return result;
}

void toString ()
{
    char result = "\n";
    for (int row = 0; row < grid.length; row++)
    {
        for (int column=0; column < grid[row].length; column++)
            result += grid[row][column] + " ";
        result += "\n";
    }
    return result;
}
```





```
void main () {  
    toString();  
  
    if (traverse (0, 0))  
        System.out.println ("The maze was successfully traversed!");  
    else  
        System.out.println ("There is no possible path.");  
  
    toString();  
}
```



Problem #2: The Towers of Hanoi

The Towers of Hanoi is a puzzle made up of three vertical pegs and several disks that slide onto the pegs. The disks are of varying size, initially placed on one peg with the largest disk on the bottom and increasingly smaller disks on top.

The goal is to move all of the disks from one peg to another following these rules:

- Only one disk can be moved at a time.

- A disk cannot be placed on top of a smaller disk.

- All disks must be on some peg (except for the one in transit).



A solution to the three-disk Towers of Hanoi puzzle



Original Configuration



Fourth Move



First Move



Fifth Move



Second Move



Sixth Move



Third Move



Seventh and Last Move



Towers of Hanoi

To move a stack of N disks from the original peg to the destination peg

Move the topmost $N-1$ disks from the original peg to the extra peg

Move the largest disk from the original peg to the destination peg

Move the $N-1$ disks from the extra peg to the destination peg

The base case occurs when a “stack” contains only one disk

Note that the number of moves increases exponentially as the number of disks increases

The recursive solution is simple and elegant to express (and program)

An iterative solution to this problem is much more complex

Towers of Hanoi

```
void main () {  
    moveTower(4, 1,3,2);  
}  
  
void moveTower (int numDisks, int start, int end, int temp)    {  
    if (numDisks == 1)  
        moveOneDisk (start, end);  
    else    {  
        moveTower (numDisks-1, start, temp, end);  
        moveOneDisk (start, end);  
        moveTower (numDisks-1, temp, end, start);  
    }  
}  
  
void moveOneDisk (int start, int end)    {  
    System.out.println ("Move one disk from " + start + " to "  
+ end);  
}
```



Exercise

1. What does the following program compute?

```
int f(int n, int v) {  
    if (n == 0)  
        return v;  
    else  
        return f(n - 1, v + n);  
}  
  
int main() {  
    return f(1000000, 0);  
}
```



Exercise

2. Using recursive technique in order to check value X is a member of list L .
3. Using recursive technique in order to double every element of a list of numbers.
4. Assume there are n types of money with different values. Pay the amount k by using n types of money above.



A bit of humor



Any Questions?



✉ hvusynh@hcmiu.edu.vn